



归并排序

基本思想

后续关注公众号[研大]
永久微信1951657888

假设初始序列含有 n 个记录，则可将其看成 n 个有序的子序列，每个子序列的长度为1，然后两两归并，得到 $\lceil n/2 \rceil$ 个长度为2或1的有序子序列；
再两两归并，如此重复，直至得到一个长度为 n 的有序序列为止



归并排序



基本思想

假设初始序列含有 n 个记录，则可将其看成 n 个有序的子序列，每个子序列的长度为1，然后两两归并，得到 $\lceil n/2 \rceil$ 个长度为2或1的有序子序列；再两两归并，如此重复，直至得到一个长度为 n 的有序序列为止

逻辑演示

扫码关注公众号
永久微信1981087888

13 27 38 49 65 76 97

经过三趟两路归并，排序完成



归并排序



```
void MergeSort(SqList &L) {
// 对顺序表 L 进行归并排序
    MSort(L, L, 1, L.length);
}

void MSort(RedType R[], RedType T[], int low, int high)
{
    // R[low..high] 归并排序后放入 T[low..high] 中
    if (low == high)
        T[low] = R[low];
    else{
        mid = (low + high) / 2;
        // 将当前序列一分为二, 求出分裂点 mid
        MSort(R, S, low, mid);
        // 对子序列 R[low..mid] 递归进行归并排序
        // 结果放入 S[low..mid]
        MSort(R, S, mid + 1, high);
        // 对子序列 R[mid+1..high] 递归进行归并排序
        // 结果放入 S[mid+1..high]
        Merge(S, T, low, mid, high);
    }
}
```

```
void Merge(RedType R[], RedType T[], int low, int mid, int high)
{
    i = low;
    j = mid + 1;
    k = low;
    while (i <= mid && j <= high){
        if (R[i] <= R[j])
            T[k++] = R[i++];
        else
            T[k++] = R[j++];
    }// while
    while (i <= mid)// 将剩余的R[i..mid]复制到T中
        T[k++] = R[i++];
    while (j <= high)// 将剩余的R[j..high]复制到T中
        T[k++] = R[j++];
}
```

时间复杂度 $O(n\log_2 n)$

空间复杂度 $O(n)$

是稳定的排序算法





基数排序

基本思想

后续关注公众号[研大]
永久微信1951657888

不比较关键字的大小，根据关键字中各位的值，对排序记录进行若干趟“分配”和“收集”实现排序的

假设关键字均为 $[0,999]$ 之间的关键字，先按照个位数排序，再按照十位数排序，最后按百位数排序



基数排序



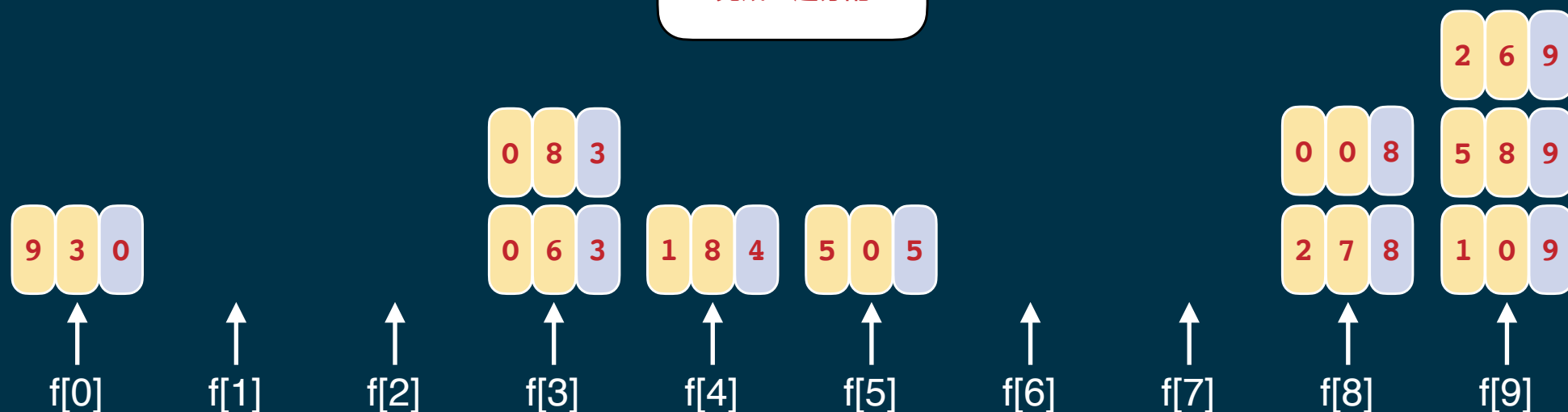
基本思想

不比较关键字的大小，根据关键字中各位的值，对排序记录进行若干趟“分配”和“收集”实现排序的
假设关键字均为[0,999]之间的关键字，先按照个位数排序，再按照十位数排序，最后按百位数排序

逻辑演示

扫码关注公众号
永久会员1981087000

完成一趟分配



基数排序



基本思想

不比较关键字的大小，根据关键字中各位的值，对排序记录进行若干趟“分配”和“收集”实现排序的
假设关键字均为[0,999]之间的关键字，先按照个位数排序，再按照十位数排序，最后按百位数排序

逻辑演示



基数排序

基本思想



不比较关键字的大小，根据关键字中各位的值，对排序记录进行若干趟“分配”和“收集”实现排序的
假设关键字均为[0,999]之间的关键字，先按照个位数排序，再按照十位数排序，最后按百位数排序

逻辑演示

依次出队，完成一次收集

永久微信1951657888



再按照十位数进行分配



基数排序

基本思想



不比较关键字的大小，根据关键字中各位的值，对排序记录进行若干趟“分配”和“收集”实现排序的
假设关键字均为[0,999]之间的关键字，先按照个位数排序，再按照十位数排序，最后按百位数排序

逻辑演示

永久微信1951657888

完成十位数分配
接下来再做一次十位数收集

1 0 9
0 0 8
5 0 5

↑
f[0]

↑
f[1]

↑
f[2]

9 3 0

↑
f[3]

↑
f[4]

↑
f[5]

2 6 9
0 6 3

↑
f[6]

2 7 8

↑
f[7]

5 8 9
1 8 4
0 8 3

↑
f[8]

↑
f[9]



基数排序



基本思想

不比较关键字的大小，根据关键字中各位的值，对排序记录进行若干趟“分配”和“收集”实现排序的
假设关键字均为[0,999]之间的关键字，先按照个位数排序，再按照十位数排序，最后按百位数排序

逻辑演示

扫码关注公众号
永久微信1981087000



基数排序

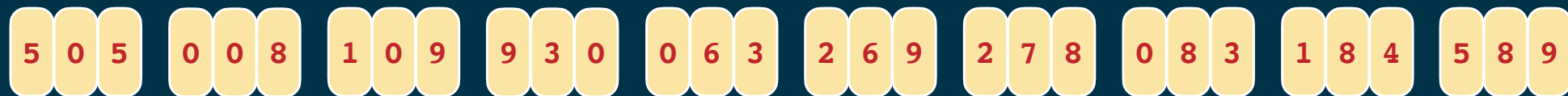


基本思想

不比较关键字的大小，根据关键字中各位的值，对排序记录进行若干趟“分配”和“收集”实现排序的
假设关键字均为[0,999]之间的关键字，先按照个位数排序，再按照十位数排序，最后按百位数排序

逻辑演示

扫码关注公众号BOK数据结构
永久微信1981087000



完成十位数收集
接下来再做一次百位数分配



基数排序

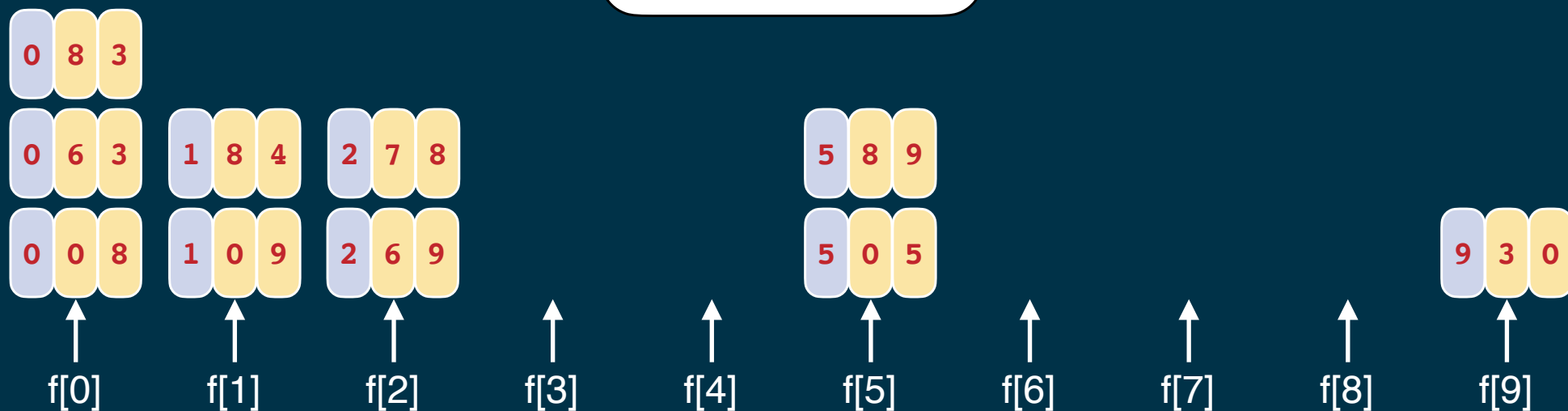
基本思想



不比较关键字的大小，根据关键字中各位的值，对排序记录进行若干趟“分配”和“收集”实现排序的
假设关键字均为[0,999]之间的关键字，先按照个位数排序，再按照十位数排序，最后按百位数排序

逻辑演示

完成百位数分配
接下来再做一次收集



基数排序



基本思想

不比较关键字的大小，根据关键字中各位的值，对排序记录进行若干趟“分配”和“收集”实现排序的
假设关键字均为[0,999]之间的关键字，先按照个位数排序，再按照十位数排序，最后按百位数排序

逻辑演示

扫码关注公众号
永久微信1981087888



基数排序

基本思想



不比较关键字的大小，根据关键字中各位的值，对排序记录进行若干趟“分配”和“收集”实现排序的
假设关键字均为[0,999]之间的关键字，先按照个位数排序，再按照十位数排序，最后按百位数排序

逻辑演示



排序完成

空间复杂度：需使用 r 个队列， $O(r)$

时间复杂度：共需 d 趟分配、收集

每趟分配需要遍历所有关键字， $O(n)$ ，

每趟收集复杂度为 $O(r)$

总复杂度为 $O(d(n + r))$

是稳定的排序算法



基数排序



```
void RadixSort(SLLList &L){
// L是采用静态链表表示的顺序表
// 对L进行基数排序, 使得L成为按关键字自小到大的有序静态链表, L.r[0]为头节点
    for (i = 0; i < L.recnum; ++i)
        L.r[i].next = i + 1;
    L.r[L.recnum].next = 0;
    // 将L改造为静态链表

    for (i = 0; i < L.keynum; ++i){
        // 按最低位优先依次对各关键字进行分配和收集
        Distribute(L.r, i, f, e); // 第i趟分配
        Collect(L.r, i, f, e); // 第i趟收集
    } // for
}
```

```
void Collect(SLCell &r, int i, ArrType f, ArrType e){
// 本算法按keys[i]自小至大地将f[0..RADIX-1]所指各子表依次链接成一个链表
// e[0..RADIX-1]为各子表的尾指针
for (j = 0; !f[j]; j = succ(j));
// 找第一个非空子表, succ()为求后继函数
r[0].next = f[j];
t = e[j];
// r[0].next指向第一个非空子表中第一个节点
while (j < RADIX){
    for (j = succ(j); j < RADIX - 1 && !f[j]; j = succ(j));
    // 找下一个非空子表
    if (f[j])
    {
        r[t].next = f[j];
        t = e[j]; // 链接两个非空子表
    }
} // while
r[t].next = 0; // t指向最后一个非空子表中的最后一个节点
}
```

```
void Distribute(SLCell &r, int i, ArrType &f, ArrType &e){
//静态链表L的r域中记录已按(keys[0], ..., keys[i-1])有序
//本算法按第i个关键字keys[i]建立RADIX个子表, 使同一子表中记录的 keys[i] 相同
//f[0..RADIX-1] 和 e[0..RADIX-1] 分别指向各子表中第一个和最后一个记录
    for (j = 0; j < RADIX; ++j)
        f[j] = 0; // 各子表初始化为空表
    for (p = r[0].next; p; p = r[p].next)
    {
        j = ord(r[p].keys[i]); //ord将记录中第i个关键字映射到[0..RADIX-1]
        if (!f[j])
            f[j] = p;
        else
            r[e[j]].next = p;
        e[j] = p; // 将p所指的节点插入第j个子表中
    } // for
}
```

排序完成

空间复杂度: 需使用r个队列, $O(r)$

时间复杂度: 共需d趟分配、收集

每趟分配需要遍历所有关键字, $O(n)$,

每趟收集复杂度为 $O(r)$

总复杂度为 $O(d(n+r))$

是稳定的排序算法

